PRACTICAL WEEK-

12 SESSION-12:

Task-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

Source code:

```
  GNU nano 8.7.1                  producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 0; i < ITEMS; i++)
    {
        /* Wait for an empty buffer slot */
        sem_wait(&empty);

        /* Enter critical section */
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        /* Increase number of full slots */
        sem_post(&full);
    }

    return NULL;
}
```

```
    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    double throughput = ITEMS / time_taken;

    printf("Buffer Size : %d\n", buffer_size);
    printf("Items       : %d\n", ITEMS);
    printf("Time        : %.6f seconds\n", time_taken);
    printf("Throughput  : %.2f items/second\n\n", throughput);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);
}

int main()
{
    printf("===== PRODUCER-CONSUMER PROBLEM =====\n\n");

    run_test(2);
    run_test(5);
    run_test(10);
    run_test(50);

    printf("===== TEST COMPLETED =====\n");

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$ nano producer_consumer.c
sathwika@DESKTOP-AA8K5MG:~$ gcc producer_consumer.c -o producer_consumer -pthread
sathwika@DESKTOP-AA8K5MG:~$ ./producer_consumer
====== PRODUCER-CONSUMER PROBLEM =====

Buffer Size : 2
Items       : 100000
Time        : 2.867784 seconds
Throughput  : 34870.13 items/second

Buffer Size : 5
Items       : 100000
Time        : 1.272477 seconds
Throughput  : 78586.86 items/second

Buffer Size : 10
Items       : 100000
Time        : 0.586819 seconds
Throughput  : 170410.31 items/second

Buffer Size : 50
Items       : 100000
Time        : 0.060096 seconds
Throughput  : 1663997.67 items/second

===== TEST COMPLETED =====
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

The Producer-Consumer problem was successfully implemented using **POSIX threads, counting semaphores, and a mutex**. Synchronization ensures correct access to the shared buffer, while changing the buffer size demonstrates its effect on system throughput.

Task-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identi%,• the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Source code:

```
 GNU nano 8.7.1                  deadlock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Trying to acquire Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Acquired Resource 1\n");
    sleep(1);

    printf("Thread 1: Trying to acquire Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Trying to acquire Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired Resource 2\n");
    sleep(1);

    printf("Thread 2: Trying to acquire Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
```

```c
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("===== DEADLOCK DEMONSTRATION =====\n\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nProgram completed.\n");

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$ nano deadlock.c
sathwika@DESKTOP-AA8K5MG:~$ gcc deadlock.c -o deadlock -pthread
sathwika@DESKTOP-AA8K5MG:~$ ./deadlock
===== DEADLOCK DEMONSTRATION =====

Thread 1: Trying to acquire Resource 1
Thread 1: Acquired Resource 1
Thread 2: Trying to acquire Resource 2
Thread 2: Acquired Resource 2
Thread 1: Trying to acquire Resource 2
Thread 2: Trying to acquire Resource 1
^C
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

The deadlock scenario was successfully demonstrated. Deadlock was prevented by imposing a **global resource-ordering rule**, where every thread acquires resources in the same predefined order.